Open in Colab

After clicking the "Open in Colab" link, copy the notebook to your own Google Drive before getting started, or it will not save your work

BYU CS 180 Lab 11: MLP and Backprop

In this lab, you will create and train a simple deep neural network. You will implement a simple DNN using the scikit-learn MLPClassifier to perform classification on the following weather dataset. Use the 'Rain' feature as the target.

Here is the documentation for the Scikit-Learn MLP Classifier https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Install kagglehub if not already installed. Uncomment and run the line below if ne
!pip install kagglehub
```

```
Requirement already satisfied: kagglehub in /usr/local/lib/python3.11/dist-packa
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packa
Requirement already satisfied: pyyaml in /usr/local/lib/python3.11/dist-packages
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-pa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/d
```

```
# Load in data
import kagglehub

# Download latest version
```



```
path = kagglehub.dataset_download("zeeshier/weather-forecast-dataset")
print(path)
df = pd.read_csv(f'{path}/weather_forecast_data.csv')
np.random.seed(42)
```

```
/kaggle/input/weather-forecast-dataset
```

```
df.head()
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

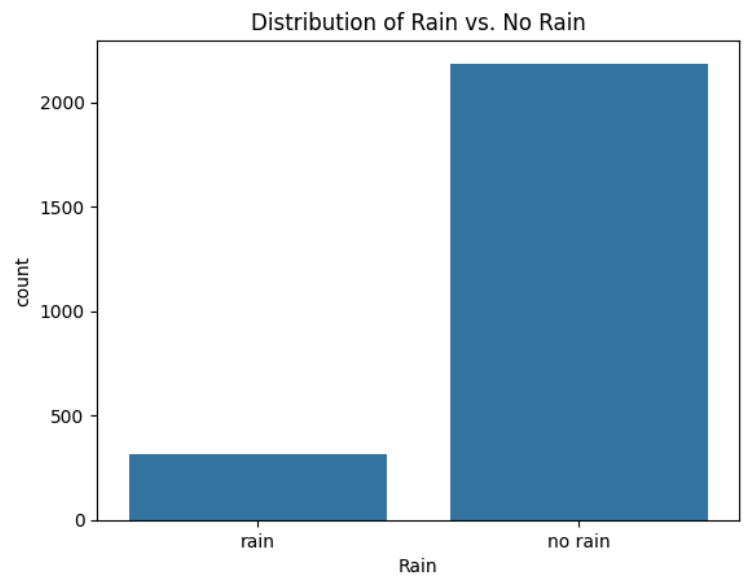
Task 1

Perform a simple exploratory analysis on the data (plot the distribution of rain vs. no rain cases, get counts for features/observations, etc.). Then split the data into a train and test set (70% / 30%), using the 'Rain' column as your target. Stratify your split. Apply any required data preparations or transformations.

```
# TODO: Simple EDA
```

```
# Distribution of rain vs. no rain
sns.countplot(x='Rain', data=df)
plt.title('Distribution of Rain vs. No Rain')
plt.show()

# Counts for features/observations
print(df.describe())
print("\n")
print(df.info())

# Get counts for each feature
for col in df.columns:
    print(f"Counts for {col}:")
    print(df[col].value_counts())
    print("\n")

df['Rain'] = pd.to_numeric(df['Rain'], errors='coerce')
```

```
# df['Rain'].fillna(0, inplace=True)
# df['Rain'] = df['Rain'].astype(int)

plt.figure(figsize=(12, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

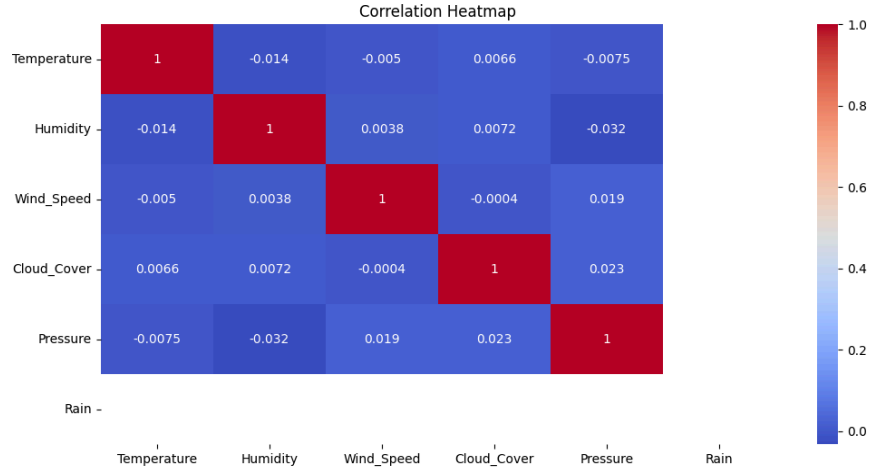
	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure
count	2500.000000	2500.000000	2500.000000	2500.000000	2500.000000
mean	22.581725	64.347094	9.906255	49.658104	1014.312336
std	7.326996	19.954739	5.780316	29.123104	20.196433
min	10.001842	30.005071	0.009819	0.015038	980.014486
25%	16.359398	47.339815	4.761909	23.900016	996.938630
50%	22.536448	63.920797	9.908572	49.488284	1013.433035
75%	28.976476	81.561021	14.948408	75.324140	1031.735067
max	34.995214	99.997481	19.999132	99.997795	1049.985593

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Temperature  2500 non-null   float64
 1   Humidity     2500 non-null   float64
 2   Wind_Speed   2500 non-null   float64
 3   Cloud_Cover  2500 non-null   float64
 4   Pressure     2500 non-null   float64
 5   Rain         2500 non-null   object
dtypes: float64(5), object(1)
memory usage: 117.3+ KB
None
Counts for Temperature:
Temperature
26.554356    1
23.720338    1
27.879734    1
```

```
25.069084    1
29.847710    1
            ..
34.091569    1
32.294325    1
20.939680    1
26.147353    1
20.591370    1
Name: count, Length: 2500, dtype: int64


Counts for Humidity:
Humidity
97.101517    1
89.592641    1
46.489704    1
83.072843    1
87.348236    1
            ..
48.057114    1
51.848471    1
40.799444    1
48.217260    1
96.858822    1
Name: count, Length: 2500, dtype: int64


Counts for Wind_Speed:
Wind_Speed
18.563084    1
7.335604     1
5.952484     1
1.371992     1
3.295808     1
            ..
5.570206     1
2.873621     1
2.232566     1
15.258547    1
4.643921     1
Name: count, Length: 2500, dtype: int64


Counts for Cloud_Cover:
Cloud_Cover
81.357508    1
50.501694    1
4.990053     1
14.855784    1
99.137103    1
            ..
82.524873    1
92.551497    1
45.827508    1
59.766279    1
47.676444    1
Name: count, Length: 2500, dtype: int64
```



```
Name: count, Length: 2500, dtype: int64


Counts for Pressure:
Pressure
1001.729176    1
1032.378759    1
992.614190     1
1007.231620    1
1016.720300    1
              ..
993.732047     1
1006.041733    1
1014.173766    1
1049.738751    1
980.825142     1
Name: count, Length: 2500, dtype: int64


Counts for Rain:
Rain
no rain    2186
rain        314
Name: count, dtype: int64
```



```
# TODO: Split Data into train and test

X = df.drop('Rain', axis=1)
y = df['Rain']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_stat
```

Suppose you are performing this analysis for a farmer who's work heavily depends on knowing if and when it is going to rain. Based on your EDA, what metric or metrics would you focus on to evaluate your model? Why?

I would focus on cloud cover and humidity, them seem to have a higher correlation to whether or not it will rain.

Task 2

Your next task is to experiment with various MLP architectures to find an optimal model. This will require playing around with different numbers and sizes of hidden layers, trying different activation functions, and finding other hyperparameters to play around with. Use the metric chosen in Task 1 on your test set as your evaluation metric.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Experiment with different MLP architectures
mlp_params = {
    'hidden_layer_sizes': [(50,), (100,), (50, 50), (100, 50)],
    'activation': ['relu', 'tanh', 'logistic'],
    'solver': ['adam', 'sgd'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate': ['constant', 'adaptive']
}
```

```
best_mlp = None
best_accuracy = 0

for hidden_layer_sizes in mlp_params['hidden_layer_sizes']:
  for activation in mlp_params['activation']:
    for solver in mlp_params['solver']:
      for alpha in mlp_params['alpha']:
```

```
    for learning_rate in mlp_params['learning_rate']:
      mlp = MLPClassifier(hidden_layer_sizes=hidden_layer_sizes, activation=acti
      mlp.fit(X_train_scaled, y_train)
      y_pred = mlp.predict(X_test_scaled)
      accuracy = accuracy_score(y_test, y_pred)

      if accuracy > best_accuracy:
          best_accuracy = accuracy
          best_mlp = mlp
          print(f"New best accuracy: {accuracy} with params: {hidden_layer_sizes

print(f"Best accuracy achieved: {best_accuracy}")
print(f"Best MLP Classifier:\n{best_mlp}")
print(classification_report(y_test, best_mlp.predict(X_test_scaled)))
```



```
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/neural_network/_multilayer_pe
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/neural_network/_multilayer_pe
  warnings.warn(
Best accuracy achieved: 0.9946666666666667
Best MLP Classifier:
MLPClassifier(max_iter=500, random_state=42)
              precision    recall  f1-score   support

     no rain       1.00      1.00      1.00       656
        rain       0.99      0.97      0.98        94

    accuracy                           0.99       750
   macro avg       0.99      0.98      0.99       750
weighted avg       0.99      0.99      0.99       750
```

What number of hidden layers worked best for you? What size were they? Why do you think this architecture worked better than others?

The best accuaracy came with one hidden layers that had 100 nuerons.

Briefly read into the different activation functions and what makes each one unique. Which one worked best for you? Why do you think that is?

Relu - fast and simple. Introduced sparsity because if a value is negative is is output as 0. values are output 0 or 1. introduces nonlinearity to model complex patterns

Tanh - zero-centered, values are between -1 and 1. effective for mapping strong positive or strong negative values.

Logistic - probablistic output, great for binary classification tasks. Learning is slower and optimization is more difficult.

Task 3

a) Fit a random forest and logistic regression to this same dataset. See how their performance compares to your best MLP. These libaries are already imported. Documentation can be found at the following links.

- https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

- https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

```
# TODO: Implement random forest


from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Fit and transform the training data
# Scale the test data using the same scaler
X_test_scaled = scaler.transform(X_test) # Transform the test data

rf = RandomForestClassifier(random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print(f"Random Forest Accuracy: {accuracy_rf}")
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 1.0
              precision    recall  f1-score   support

     no rain       1.00      1.00      1.00       656
        rain       1.00      1.00      1.00        94

    accuracy                           1.00       750
   macro avg       1.00      1.00      1.00       750
weighted avg       1.00      1.00      1.00       750
```

```
# TODO: Implement logistic regression
# TODO: Implement logistic regression
lr = LogisticRegression(random_state=42, max_iter=500)
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)
accuracy_lr = accuracy_score(y_test, y_pred_lr)
print(f"Logistic Regression Accuracy: {accuracy_lr}")
print(classification_report(y_test, y_pred_lr))
```

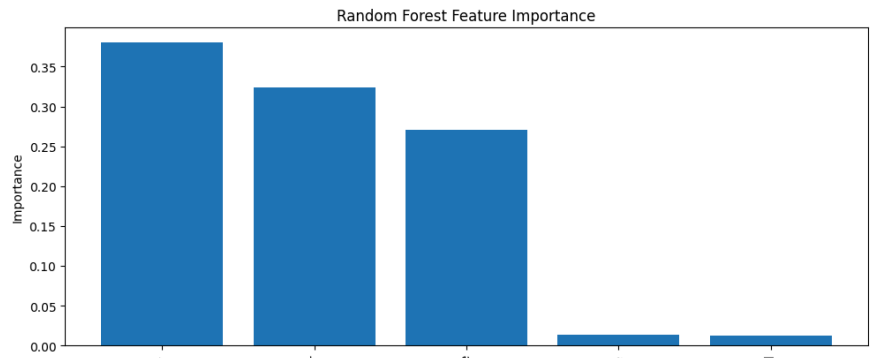
```
Logistic Regression Accuracy: 0.9186666666666666
              precision    recall  f1-score   support

     no rain       0.95      0.96      0.95       656
        rain       0.69      0.63      0.66        94

    accuracy                           0.92       750
   macro avg       0.82      0.79      0.81       750
weighted avg       0.92      0.92      0.92       750
```


b) For each model, determine a measure of variable importance. For the random forest, you can use the feature importances_ attribute. For the logistic regression, you can use the absolute value of the coefficients. For the MLP, you can use permutation importance (see documentation below).

Permutation Importance

```
from sklearn.inspection import permutation_importance

# Feature Importance for Random Forest
rf_feature_importance = rf.feature_importances_
print("Random Forest Feature Importance:")
for i, importance in enumerate(rf_feature_importance):
    print(f"{X.columns[i]}: {importance}")


# Feature Importance for Logistic Regression
lr_coefficients = np.abs(lr.coef_[0])
print("\nLogistic Regression Feature Importance (abs of coefficients):")
for i, coef in enumerate(lr_coefficients):
    print(f"{X.columns[i]}: {coef}")

# Feature Importance for MLP using Permutation Importance
result = permutation_importance(best_mlp, X_test_scaled, y_test, n_repeats=10, rando
print("\nMLP Feature Importance (Permutation Importance):")
for i in result.importances_mean.argsort()[::-1]:
    print(f"{X.columns[i]}: {result.importances_mean[i]} +/- {result.importances_std
```

```
Random Forest Feature Importance:
Temperature: 0.2702978374871589
Humidity: 0.37987843087030376
Wind_Speed: 0.011904643873467943
Cloud_Cover: 0.32395812646355415
Pressure: 0.013960961305515263

Logistic Regression Feature Importance (abs of coefficients):
Temperature: 1.8858477204303434
Humidity: 2.738855062579202
Wind_Speed: 0.030965085823364644
Cloud_Cover: 2.354114211626172
Pressure: 0.055555863962856146

MLP Feature Importance (Permutation Importance):
Humidity: 0.14013333333333336 +/- 0.007875701026998572
Cloud_Cover: 0.12493333333333337 +/- 0.008220300075625766
Temperature: 0.09973333333333337 +/- 0.006016274225428508
Wind_Speed: 0.0025333333333333653 +/- 0.0011075498483890723
Pressure: 0.0016000000000000458 +/- 0.0011623730516108456
```

c) For each set of importances, plot the feature importances using a bar plot, sorted by importance. Include each of the three plots in a single figure with three subplots. (Note: the three sets of scores will be on different scales)

```
# prompt: For each set of importances, plot the feature importances using a bar plot

import matplotlib.pyplot as plt

# Create a figure and subplots
fig, axes = plt.subplots(3, 1, figsize=(10, 15))

# Function to create bar plot
def plot_feature_importance(importances, feature_names, ax, title):
    indices = np.argsort(importances)[::-1]
    ax.bar(range(len(indices)), importances[indices], align='center')
    ax.set_xticks(range(len(indices)))
    ax.set_xticklabels(np.array(feature_names)[indices], rotation=90)
    ax.set_title(title)
    ax.set_ylabel('Importance')


# Plot Random Forest feature importances
plot_feature_importance(rf_feature_importance, X.columns, axes[0], "Random Forest Fe


# Plot Logistic Regression feature importances
plot_feature_importance(lr_coefficients, X.columns, axes[1], "Logistic Regression Fe

# Plot MLP feature importances
plot_feature_importance(result.importances_mean, X.columns, axes[2], "MLP Feature Im


plt.tight_layout()
plt.show()
```